Trabajo de Aprendizaje Supervisado:

Predicción de precios de coches

UNIVERSIDAD
COMPLUTENSE
MADRID

Marcos Martínez, Laura López, Marina Parra, Verónica Yebra y Rodrigo Castillo.

FASE 1: DEPURACIÓN DE LA BASE DE DATOS

Para este trabajo de investigación hemos seleccionado la base de datos *car_price.csv.* Este conjunto de datos contiene información detallada sobre vehículos usados en venta en India. Con este conjunto de datos buscamos predecir el precio de los coches usados y ver cómo afectan las variables que vamos a utilizar. Para ello primeramente realizamos un proceso de depuración y recodificación de la base de datos.

- ## CODIFICACIÓN DE VARIABLES CUALITATIVAS

```{r}
str(datos)
```

```
'data.frame':   7400 obs. of  16 variables:
 $ yr_mfr            : int  2015 2016 2017 2013 2015 2018 2012 2012 2014 2018 ...
 $ fuel_type         : Factor w/ 5 levels "diesel","electric",..: 3 3 3 1 3 3 1 3 1 3 ...
 $ kms_run           : num  8063 23104 23402 39124 22116 ...
 $ sale_price        : num  386399 265499 477699 307999 361499 ...
 $ body_type         : Factor w/ 5 levels "hatchback","luxury sedan",..: 1 1 1 1 1 1 1 1 1 1 ...
 $ transmission      : Factor w/ 2 levels "automatic","manual": 2 2 2 2 2 NA 2 2 2 2 ...
 $ assured_buy       : logi  TRUE TRUE TRUE TRUE FALSE TRUE ...
 $ registered_city   : Factor w/ 243 levels "-do","agra","ahmedabad",..: 66 160 2 66 158 66 66 160 160 160
...
 $ is_hot            : logi  TRUE TRUE TRUE TRUE TRUE TRUE ...
 $ make              : Factor w/ 27 levels "audi","bmw","chevrolet",..: 15 15 8 15 8 15 15 8 8 21 ...
 $ total_owners      : int  2 1 1 1 1 1 1 3 1 1 ...
 $ original_price    : int  404177 354313 NA 374326 367216 439056 NA 410764 566123 344127 ...
 $ car_rating        : Factor w/ 4 levels "fair","good",..: 3 3 3 3 3 3 3 3 3 3 ...
 $ ad_created_on     : Factor w/ 7398 levels "2019-02-20 14:13:30.464",..: 6011 4733 4400 1216 5810 6751 995
2342 3751 4414 ...
 $ fitness_certificate: logi  TRUE TRUE TRUE TRUE TRUE TRUE ...
 $ warranty_avail    : logi  FALSE FALSE FALSE FALSE FALSE FALSE ...
```

Podemos comprobar que las variables **make** y **registered_city** cuentan con 27 y 243 categorías respectivamente por lo que las reagrupamos. También contamos con una variable fecha "**ad_created_on**" a la que realizaremos un tratamiento diferente más adelante.

Como se puede comprobar en las fotografías, hemos recodificado las variables para que en la variable make (fabricante del coche) pase a llamarse **"casa"** haciendo referencia a la casa del motor del coche (Toyota, Volkswagen_Group, etc.) y así hemos conseguido pasar de 27 a 14 categorías. De este modo, con registered_city que hacía referencia a la ciudad en la que estaba registrado el coche que contaba con 244 categorías, ahora se ha agrupado en base a 5 regiones de la India (Norte, Sur, Este, Oeste o Centro).

De la variable fecha "ad_created_on", se han extraído los meses días y año para poder trabajar con ella dando lugar a tres nuevas variables: **year_anuncio**, que es el año en el que se creó el anuncio de la venta, **mes_anuncio**, que cuenta con mes en el que se creó el anuncio de la venta y **dia_anuncio** que es el día del mes en el que se creó el anuncio de la venta.
Además, se han pasado a factor todas las variables cualitativas.

- ## CODIFICACIÓN DE VARIABLES CUANTITATIVAS

En este punto encontramos un error en la variable objetivo (sale_price), puesto que tomaba un valor mínimo de 0 y no es posible que el precio de venta final de un coche sea de 0 unidades monetarias, por ello, vamos a quitar esta observación entera. Además, como en la variable original_price casi la mitad de la información son missings vamos a prescindir de ella.

- ## OUTLIERS Y VALORES AUSENTES

En este apartado se ha realizado un estudio univariante y bivariante de los datos atípicos para tomar una decisión acerca de su tratamiento. Tras emplear la función:
outliers_results ← lapply(numeric_integer_vars, function(var) {outliers(datos, var)})

Observamos que las variables que presentan outliers son **yr_mfr** con un 0.54% de outliers, **kms_run** con un 2.41%, **sale_price** con un 5.57%, **total_owners** con el 0.58%, year_anuncio` con el 13.73% de outliers y **mes_anuncio** con el 15.24% de outliers.

Podemos observar que las variables **year_anuncio**, **sale_price** y **mes_anuncio** presentan una gran cantidad de outliers por lo que vamos a descartar que son atípicos y vamos a considerar que estas variables tienen una distribución rara, ya que si eliminásemos esos valores, estaríamos perjudicando bastante información de la base de datos.

Por otro lado, como **yr_mfr** y **total_owners** presentan un porcentaje menor al 1% de outliers, vamos a tratarlos como atípicos y eliminarlos de la base de datos. En cambio, con **kms_run** que era la que presentaba un porcentaje más dudoso a la hora de tomar una decisión, se realizó el análisis bivariante y se vio que las observaciones de los coches que tenían más de 500.000 kilómetros eran verdaderamente outliers y por tanto los pasamos a NA.

En cuanto a los valores ausentes, la base de datos contaba con un porcentaje total de 0.60% de missings, lo que da lugar a un 91.4% de observaciones completas (8.6% afectadas por los missings). Dado que es un porcentaje muy bajo, vamos a eliminar las observaciones que contienen valores ausentes.

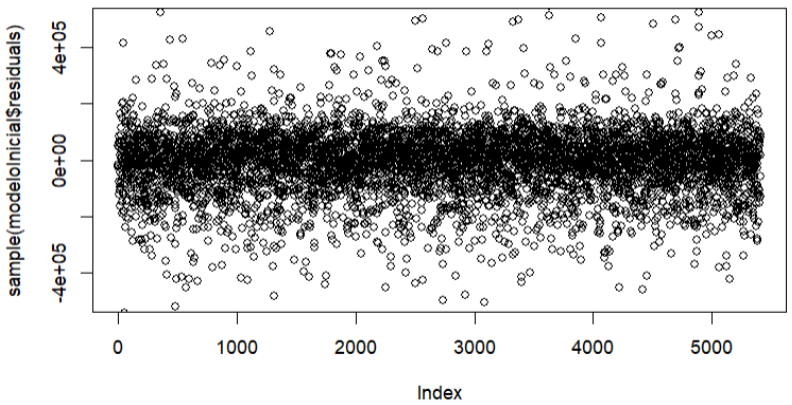
FASE 2: MODELIZACIÓN

2.1 MODELO GLM

En este apartado, dado que nuestra variable objetivo es cuantitativa, realizaremos una modelo de regresión lineal.

Es importante destacar que, dado que contamos con muchas observaciones (6760 observaciones), por las propiedades de los estimadores de los parámetros, la normalidad asintótica está asegurada, por lo que sólo sería necesario verificar la hipótesis de homocedasticidad para poder confiar el los p-valores posteriores. Para ello, se crea un **modelo_inicial** con todas las variables regresoras y se comprueba gráficamente la homocedasticidad.

No se observa ningún patrón, por lo que podemos dar por buena la hipótesis de homocedasticidad de los residuos y confiar en los p-valores que nos ofrezcan las funciones de este modelo.



Este **modelo_inicial** contaba con 40 parámetros y un R^2 en entrenamiento de 0.7064 y en prueba de 0.7037. No obstante, a pesar de no haber sobreajuste, conviene siempre analizar si alguna variable regresora puede ser rechazada, por no aportar información al modelo.

Por ello, continuamos con el **análisis de tipo II** de las variables, para poder determinar cuáles podríamos eliminar del modelo:

```{r}
Anova(modeloInicial, type = "II")
```

```
Anova Table (Type II tests)

Response: sale_price
                     Sum Sq   Df   F value    Pr(>F)
yr_mfr             3.8797e+13    1 1875.4759 < 2.2e-16 ***
fuel_type          8.3097e+12    4  100.4239 < 2.2e-16 ***
kms_run            1.3095e+12    1   63.2996 2.149e-15 ***
body_type          4.0574e+13    4  490.3442 < 2.2e-16 ***
transmission       6.9810e+12    1  337.4678 < 2.2e-16 ***
assured_buy        7.5423e+10    1    3.6460 0.0562579 .
is_hot             2.8943e+11    1   13.9914 0.0001856 ***
total_owners       4.2098e+11    1   20.3503 6.586e-06 ***
car_rating         2.8634e+11    3    4.6139 0.0031530 **
fitness_certificate 7.0482e+10    1    3.4071 0.0649705 .
warranty_avail     1.2995e+10    1    0.6282 0.4280526
casa               2.3970e+13   13   89.1341 < 2.2e-16 ***
region             5.9306e+12    4   71.6725 < 2.2e-16 ***
year_anuncio       6.3128e+11    1   30.5164 3.466e-08 ***
mes_anuncio        3.5697e+11    1   17.2562 3.317e-05 ***
dia_anuncio        2.4781e+11    1   11.9794 0.0005421 ***
Residuals          1.1105e+14 5368
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1
```

Observamos que las variables no significativas son assured_buy (aunque está muy al límite de significancia), fitness_certificate y warranty_avail.

Por esta razón, se genera un modelo denominado **modelo2** con solamente las variables significativas.

Frente a este modelo se recurrió a la función **add1( )** que evalúa la disminución de la SSE que tiene lugar cuando se añade una nueva variable y obtuvimos lo siguiente:

```{r}
add1(modelo2, scope=formula(modeloInicial),test = "F")
```

```
Single term additions

Model:
sale_price ~ (yr_mfr + fuel_type + kms_run + body_type + transmission +
    assured_buy + is_hot + total_owners + car_rating + fitness_certificate +
    warranty_avail + casa + region + year_anuncio + mes_anuncio +
    dia_anuncio) - warranty_avail - fitness_certificate - assured_buy
                    Df  Sum of Sq        RSS    AIC F value  Pr(>F)
<none>                            1.1122e+14 128497
assured_buy          1 9.0383e+10 1.1113e+14 128495  4.3675 0.03668 *
fitness_certificate  1 7.8772e+10 1.1114e+14 128495  3.8060 0.05112 .
warranty_avail       1 2.3429e+10 1.1120e+14 128498  1.1314 0.28752
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1
```

Frente a este resultado vimos que Variable assegured_by que habíamos eliminado sale significativa por lo que la eliminación de assured_buy no ha sido acertada por lo que la incluimos nuevamente en el modelo. A este nuevo modelo con la variable assegured_by lo hemos llamado **modelo2Def.**

De estas forma, obtenemos las métricas para el **modelo2Def:**

```{r}
modelo2Def <- lm(sale_price~.-warranty_avail-fitness_certificate, data=datos_train)
R2(predict(modelo2Def,datos_train), datos_train$sale_price)
```

```
[1] 0.7062125
```

```{r}
R2(predict(modelo2Def,datos_test), datos_test$sale_price)
```

```
[1] 0.7032461
```

```{r}
modelo2Def$rank
```

```
[1] 38
```

Vemos que el **modelo2Def** ha conseguido un R^2 en entrenamiento de 0.706 y en prueba de 0.703 lo que nos afianza la idea de que hemos conseguido un modelo estable con apenas sobre ajuste aún teniendo en cuenta la gran cantidad de parámetros con la que cuenta el modelo. Es decir, nuestro modelo está siendo capaz de generalizar los resultados, hemos conseguido que el modelo " aprenda" y no "memorice".

- **INTERPRETACIÓN DE COEFICIENTES:**

Hemos realizado la interpretación de algunos parámetro

- **yr_mfr= 3.831e+04**: el parámetro es positivo, por lo que por cada año, mayor colesterol. Concretamente, un incremento de un año en el año de fabricación implica un aumento de 38310 unidades monetarias.

- **fuel_typeelectric= -2.277e+05**: la categoría de referencia es *diesel*. Los coches eléctricos tiene un precio de 227700 unidades monetarias inferior que los coches diesel.

- **kms_run= -4.704e-01**: el parámetro es negativo por lo que por cada kilómetro demás recorrido por el coche, su precio disminuye 0.47 unidades monetarias (50 céntimos aproximadamente).

- **body_typeluxury sedan= 3.731e+05**: la categoría de referencia es *hatchback*. Los coches con una carrocería luxury sedan, tienen un precio de 373100 unidades monetarias mayor que los coches con carrocería hatchback.

- **is_hotTRUE= -6.232e+04**: Si el coche es un anuncio "destacado" (alta demanda) presentan un precio 62320 unidades monetarias inferior a los coches sin anuncio destacado.

- **total_owners = -1.696e+0**4: por cada dueño de más que ha tenido un coche, el precio de este disminuye 16960 unidades monetarias.

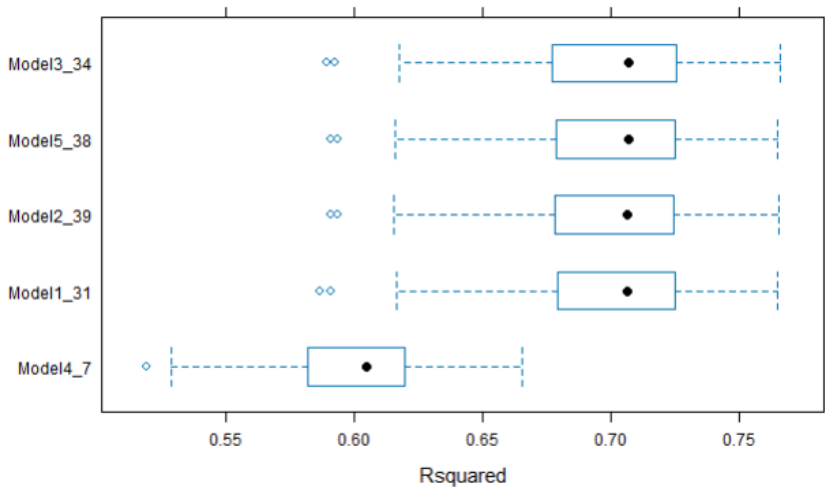
- **car_ratinggood = 1.711e+04:** la categoría de referencia es *fair*. Los coches que han obtenido una calificación "good" tienen un precio 17110 unidades monetarias superior al de los coches calificados como fair.

- **casaFord = -2.858e+05:** la categoría de referencia es *BMW_Group*. Los coches ford tienen un precio 28580 unidades monetarias inferior que los BMW.

- **regionEste = -1.162e+05**: la categoría de referencia es centro. Los coches del este de la India tiene un precio de 11620 unidades monetarias inferior que los coches del centro de la India.

MÉTODOS AUTOMÁTICOS DE SELECCIÓN DE VARIABLES

Para realizar la selección automática de variables hemos definido un **modeloSencillo** con las tres variables más significativas. En el ANOVA con todas las variables vimos que las tres de mayor significancia para nuestra variable objetivo son **yr_mfr** (F value = 1875.3782) seguida de **body_type** (490.1264) y **transmission** (338.4417).




```{r}
summary(resamples(vcrTodosModelos), metric=c("Rsquared"))
```

```
Call:
summary.resamples(object = resamples(vcrTodosModelos), metric = c("Rsquared"))

Models: Model1_31, Model2_39, Model3_34, Model4_7, Model5_38
Number of resamples: 100

Rsquared
           Min.   1st Qu.    Median      Mean   3rd Qu.      Max. NA's
Model1_31 0.5870418 0.6793404 0.7063565 0.6975105 0.7250148 0.7648312    0
Model2_39 0.5911640 0.6787563 0.7067381 0.6981383 0.7244477 0.7654237    0
Model3_34 0.5894424 0.6778623 0.7073730 0.6983136 0.7251605 0.7660665    0
Model4_7  0.5191709 0.5826463 0.6049905 0.6008428 0.6194909 0.6653970    0
Model5_38 0.5911704 0.6793927 0.7072186 0.6986647 0.7249539 0.7653192    0
```

A la hora de tomar una decisión respecto a qué modelo es el "ganador", debemos tener en cuenta tres aspectos que son:

En primer lugar, la **capacidad predictiva**, que en regresión lineal lo mediremos con el R^2. En este caso podemos observar que los 3 primeros modelos presentan valores muy similares que rondan el 70% mientras que el model4_7 tiene una clara peor calidad predictiva. Por tanto, descartamos este último como ganador.

En segundo lugar, atendemos a la **estabilidad** de estos modelos. Cuanto **más cortos sean los bigotes y más estrechas sean las cajas, más estables** son los modelos, ya que esto indica que todos los resultados de las iteraciones son "similares" (hay menor dispersión entre las iteraciones). De nuevo, de los tres restantes volvemos a obtener resultados muy similares en cuanto a la estabilidad.

En tercer lugar, buscamos **modelos parsimoniosos**, es decir, que sean lo más sencillos posibles para poder optar a una interpretación más clara y sencilla. En este caso, el modeloStepBIC y el modeloBackBIC son los que tienen un menor número de parámetros.

```{r}
R2(predict(modeloStepBIC,datos_train), datos_train$sale_price)
```

```
[1] 0.7033951
```

```{r}
R2(predict(modeloStepBIC,datos_test), datos_test$sale_price)
```

```
[1] 0.7043514
```

```{r}
modeloStepBIC$rank
```

```
[1] 31
```

```{r}
Anova(modeloStepBIC,type = "II")
```

```
Anova Table (Type II tests)

Response: sale_price
                Sum Sq   Df  F value    Pr(>F)
body_type    4.0875e+13    4  489.732 < 2.2e-16 ***
yr_mfr       4.1968e+13    1 2011.303 < 2.2e-16 ***
casa         2.3566e+13   13   86.878 < 2.2e-16 ***
transmission 7.0040e+12    1  335.668 < 2.2e-16 ***
fuel_type    7.9979e+12    4   95.825 < 2.2e-16 ***
region       6.3196e+12    4   75.717 < 2.2e-16 ***
kms_run      1.3838e+12    1   66.318 4.730e-16 ***
total_owners 3.7908e+11    1   18.167 2.058e-05 ***
dia_anuncio  1.8274e+11    1    8.758  0.003096 **
Residuals    1.1220e+14 5377
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1
```

```
Coefficients:
                        Estimate Std. Error t value Pr(>|t|)
(Intercept)           -7.469e+07  1.686e+06 -44.289  < 2e-16 ***
body_typeluxury sedan  3.759e+05  1.863e+04  20.181  < 2e-16 ***
body_typeluxury suv    5.374e+05  1.422e+04  37.803  < 2e-16 ***
body_typesedan         6.591e+04  5.739e+03  11.484  < 2e-16 ***
body_typesuv           1.960e+05  6.931e+03  28.277  < 2e-16 ***
yr_mfr                 3.750e+04  8.363e+02  44.848  < 2e-16 ***
casaDaimler            3.579e+05  3.979e+04   8.994  < 2e-16 ***
casaFord              -2.742e+05  3.331e+04  -8.233 2.28e-16 ***
casaGeely_Group        6.380e+05  1.474e+05   4.330 1.52e-05 ***
casaGM                -3.463e+05  3.858e+04  -8.974  < 2e-16 ***
casaHonda_Group       -1.760e+05  3.191e+04  -5.514 3.66e-08 ***
casaHyundai_Group     -1.827e+05  3.124e+04  -5.846 5.32e-09 ***
casaNissan_Alliance   -3.528e+05  3.582e+04  -9.848  < 2e-16 ***
casaOtros             -2.372e+05  3.110e+04  -7.626 2.84e-14 ***
casaRenault_Group     -3.605e+05  3.296e+04 -10.937  < 2e-16 ***
casaStellantis        -1.553e+05  4.382e+04  -3.544 0.000398 ***
casaTata_Motors       -2.889e+05  3.402e+04  -8.492  < 2e-16 ***
casaToyota_Group      -2.201e+04  3.123e+04  -0.705 0.481048
casaVolkswagen_Group  -1.782e+05  3.134e+04  -5.686 1.37e-08 ***
transmissionmanual    -1.347e+05  7.353e+03 -18.321  < 2e-16 ***
fuel_typeelectric     -2.184e+05  7.263e+04  -3.007 0.002654 **
fuel_typepetrol       -9.420e+04  5.592e+03 -16.844  < 2e-16 ***
fuel_typepetrol & cng -1.430e+05  9.519e+03 -15.022  < 2e-16 ***
fuel_typepetrol & lpg -7.961e+04  2.798e+04  -2.845 0.004458 **
regionEste            -1.015e+05  3.034e+04  -3.347 0.000824 ***
regionNorte           -6.259e+04  4.824e+03 -12.975  < 2e-16 ***
regionOeste           -2.581e+04  7.111e+03  -3.630 0.000286 ***
regionSur              4.404e+04  6.023e+03   7.312 3.03e-13 ***
kms_run               -4.839e-01  5.942e-02  -8.144 4.73e-16 ***
total_owners          -1.620e+04  3.801e+03  -4.262 2.06e-05 ***
dia_anuncio            6.732e+02  2.275e+02   2.959 0.003096 **
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1
```

Por tanto, el modelo **ganador** es modeloStepBIC ya que obtiene un R^2 ligeramente superior al modeloBackBIC y en cuanto a estabilidad y capacidad predictiva obtiene resultados muy similares al resto.

Nuestro modelo ganador **“modeloStepBic”** cuenta con un R^2 de 0.703 en entrenamiento y 0.704 en prueba y 31 parámetros con lo que podemos concluir lo mismo que habíamos visto para el modelo2Def, hemos conseguido un modelo estable con apenas sobre ajuste (teniendo en cuenta la gran cantidad de parámetros con la que cuenta el modelo). Es decir, nuestro modelo está siendo capaz de generalizar los resultados, hemos conseguido que el modelo “ aprenda” y no “memorice”.

Interpretaremos nuevamente los coeficientes de este último modelo ganador.

- **body_typeluxury sedan= 3.759e+05**: la categoría de referencia es *hatchback*. Los coches con una carrocería luxury sedan, presentan un precio aproximadamente 375900 unidades monetarias superior al de los vehículos con carrocería hatchback.

- **yr_mfr= 3.750e+04**: el parámetro es positivo, por lo que por cada año, mayor colesterol. Concretamente, un incremento de un año en el año de fabricación implica un aumento de 37500 unidades monetarias.

- **casaFord = -2.742e+05**: la categoría de referencia es *BMW_Group*. Los coches Ford tienen un precio 274200 unidades monetarias inferior que los BMW.

- **transmissionmanual = -1.347e+05**: la categoría de referencia es *automatic*. Los coches manuales tienen un precio de 134700 unidades monetarias inferior que los coches automáticos.

- **fuel_typeelectric= -2.184e+05**: la categoría de referencia es *diesel*. Los coches eléctricos tienen un precio de 218400 unidades monetarias inferior que los coches diésel.

- **regionNorte = -6.259e+04**: la categoría de referencia es *centro*. Los coches del norte de la India tienen un precio 62590 unidades monetarias inferior que los coches del centro de la India.

- **kms_run= -4.839e-01**: el parámetro es negativo por lo que por cada kilómetro de más recorrido por el coche, su precio disminuye 0.48 unidades monetarias (50 céntimos aproximadamente).

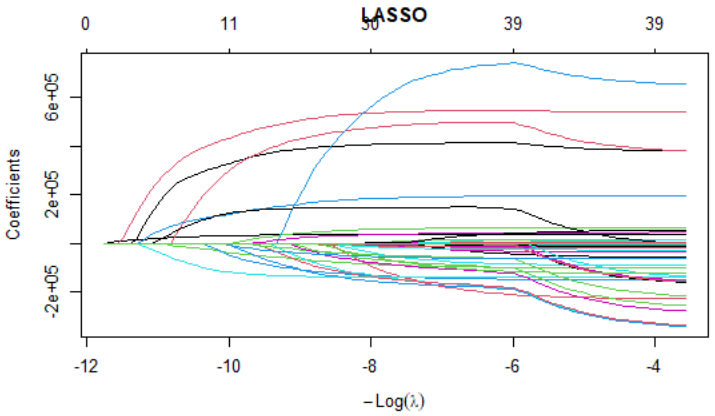
- **total_owners = -1.620e+04**: por cada dueño de más que ha tenido un coche, el precio de este disminuye 16200 unidades monetarias.

REGRESIÓN PENALIZADA

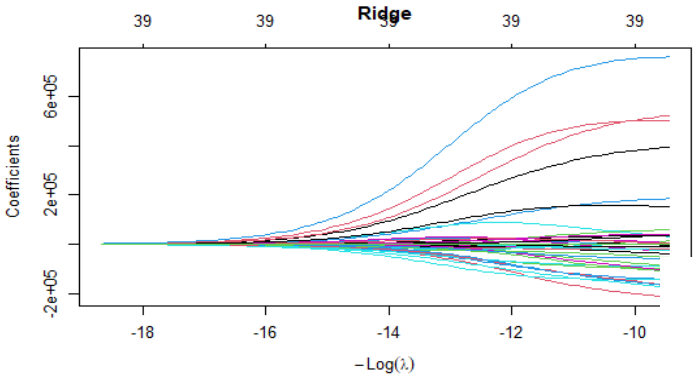
Aunque los datos con los que estamos trabajando no sufren ninguno de los problemas como un número bajo de observaciones o multicolinealidad, vamos a ilustrar la creación de estos modelos, lo que puede utilizarse como un método de selección de variables.

A continuación, vamos a proceder a crear un modelo LASSO (alpha = 1), uno Ridge (alpha = 0) y uno ENET (alpha entre 0,1) para ilustrar tanto su construcción como su funcionamiento:

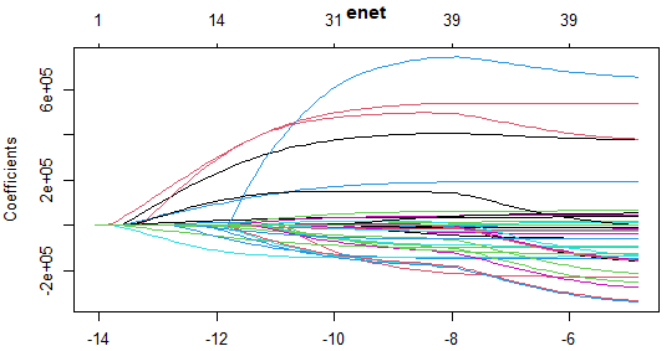
En primer lugar, consiste en minimizar los errores al cuadrado añadiendo una determinada penalización para castigar los coeficientes grandes y así evitar el sobreajuste.



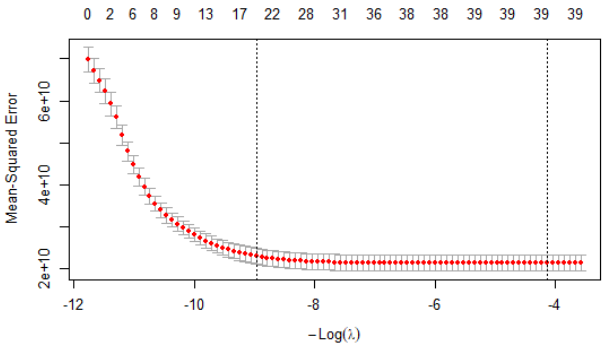
A medida que la penalización aumenta, la magnitud de los parámetros va disminuyendo. Hay un punto en el que la penalización es tan grande que todos los coeficientes llegan a 0, cosa que no ocurre en ridge (que nunca llegan a 0).



Puede parecer que el modelo ridge es más agresivo, pues los parámetros descienden "antes", pero lo que ocurre es que nunca llegan a valer cero absoluto, como sí ocurre en los otros dos casos.

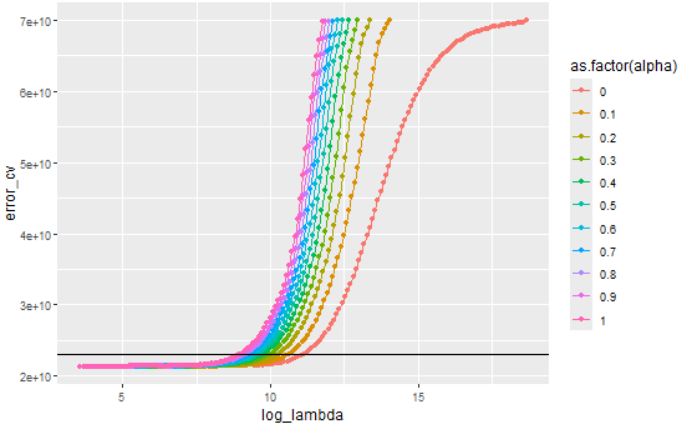


El modelo enet muestra un perfil intermedio entre lasso y ridge.
A continuación, comenzaremos afinando un modelo lasso:



En este primer gráfico podemos describir varias cosas. En el eje y se ven los ECM de todos los modelos creados en la secuencia, marca con un punto rojo la media de los 5 resultados y con bigotes, la desviación típica. La línea vertical de la derecha es el valor óptimo de landa mientras que la línea vertical de la izquierda es el valor que viene dado por la regla 1se, el cual es más parsimonioso.

No existen grandes diferencias entre los errores medios, por lo que el principio de parsimonia nos indicaría quedarnos con el modelo más sencillo.



Excepto del modelo con alpha = 0, y las mayores penalizaciones, no hay grandes diferencias entre los modelos probados.

La línea horizontal marca el límite de la regla 1se que estará alrededor del valor 2.5e+10. El mejor resultado implica un ECM de alrededor de 2e+10 Muchos modelos quedan por debajo por lo que escogeremos aquella combinación de parámetros que implique una cantidad menor de parámetros no nulos.

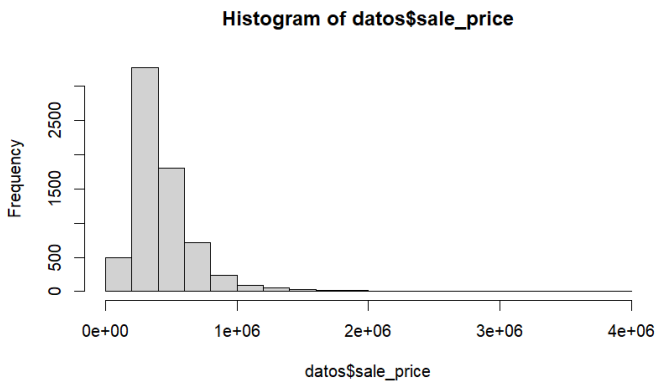
	alpha <dbl>	lambda <dbl>	log_lambda <dbl>	numPar <int>	error_cv <dbl>	error_sd <dbl>	error_up <dbl>
s30...1	1.0	7843.857	8.967486	20	23042074404	2191400185	25233474589
s30...2	0.9	8715.396	9.072846	20	23064730528	2194228947	25258959475
s30...3	0.8	9804.821	9.190629	20	23093811185	2197824791	25291635976

Podemos observar que hay 3 combinaciones de parámetros que dan lugar a modelos con 20 parámetros no nulos (sin contar con el término independiente). Cualquiera de ellas podría ser una solución razonable pero como queremos reducir al máximo el sobreajuste, elegiremos el que aparece en primera posición pues da lugar a un menor número de parámetros en la combinación.

Train: 0.6848784 Test: 0.6796265

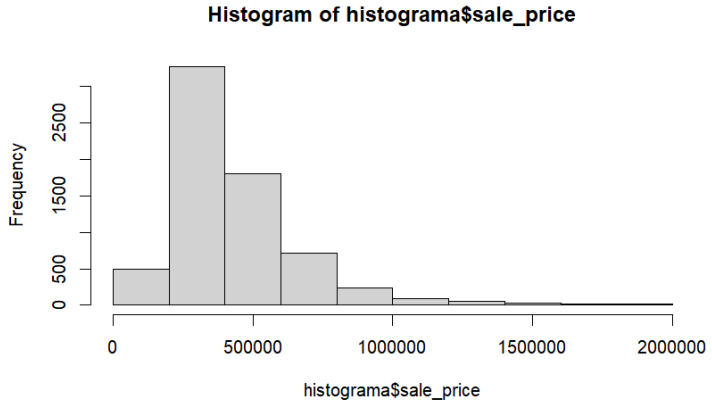
El sobreajuste se ha mantenido constante pero los resultados son algo peores, respecto a regresión lineal han bajado en torno a 0.03 puntos.

2.2 MODELOS DE ÁRBOL



Antes de empezar, hemos realizado un histograma, para ver cómo se distribuyen los datos del precio.

Como no se podían apreciar del todo bien, decidimos limitar el histograma poniendo un precio máximo de 2.000.000 u.m. ya que al poseer datos tan altos, no se observa bien su distribución.



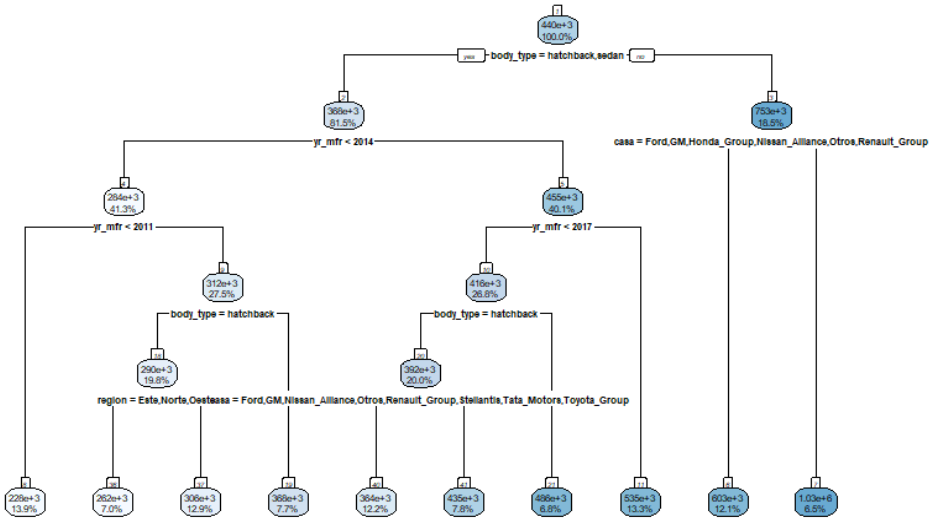
En el histograma se puede ver que la mayoría de los precios están en torno a 250000/30000,que corresponden a unos 3500 datos y la variable tiene una media de 454889, mostrada en el summary del análisis exploratorio.

Creación de los modelos

A continuación, vamos a crear varios modelos de árbol para poder contrastarlos y elegir el mejor para la poda. Comenzaremos con un tamaño mínimo de la hoja del 5%.

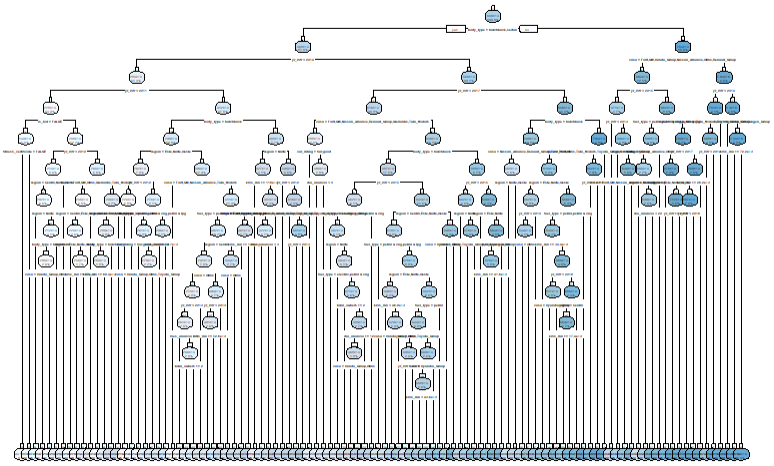
Todos los árboles se interpretan de la misma forma, por lo que para abreviar esta parte comentaremos este y el modelo resultante podado.

Hemos obtenido un árbol con 10 hojas y una profundidad de 5 en el cual se puede observar que el nodo con media más baja es el 8, que se encuentra en la esquina izquierda, con un valor de 228000 y está formado por coches con carrocería hatchback o sedán y un año de fabricación inferior a 2011. El nodo con el valor más alto es el



7, dibujado con azul oscuro, con un valor de predicción de 1031522.45142857 que corresponden al 6.5% de observaciones, que son coches con carrocería luxury sedan,luxury suv y suv.

Pasamos a la creación de los otros modelos, con un tamaño de hoja del 2%, el resultado es un árbol de 27 hojas y una profundidad de 7. Continuamos con un tamaño de hoja del 1%, con 53 hojas y una profundidad de 10.
Por último, un tamaño de hoja del 0.5%, con 106 hojas y una profundidad de 13. Se puede observar cómo a medida que se disminuye el tamaño, el árbol se hace más complejo y difícil de interpretar.



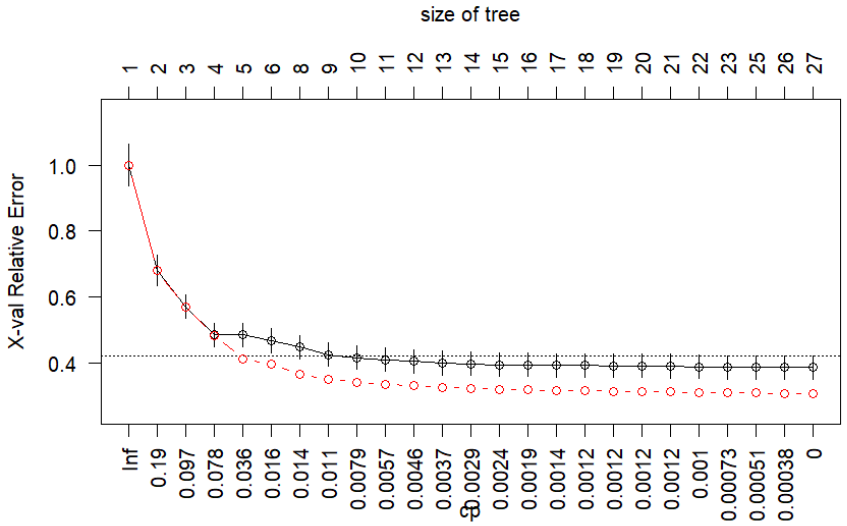
Selección del modelo

Para poder seleccionar el modelo más adecuado, nos tenemos que fijar en el sobreajuste, la calidad de las predicciones y su estabilidad. Para ello hemos utilizado el R2 que nos ayuda a sacar las conclusiones que buscamos.

	Modelo 5	Modelo 2	Modelo 1	Modelo 05
Train	0.5580887	0.6915998	0.7390952	0.7781568
Test	0.5471700	0.6711375	0.7117095	0.7467122
Nº hojas	10	27	53	106

El modelo 2 es el que menos sobreajuste tiene, con una buena predicción para el número de hojas que posee, ya que 27 hojas lo consideramos como una buena cantidad a la hora de seleccionar el modelo para la poda.

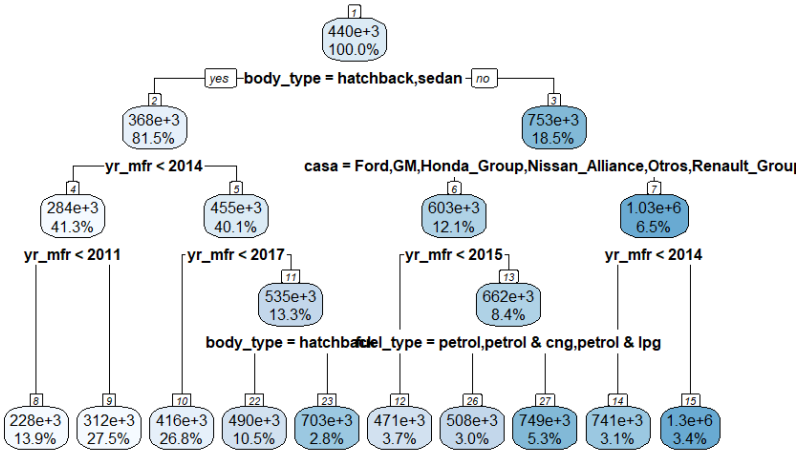
Poda del modelo 2



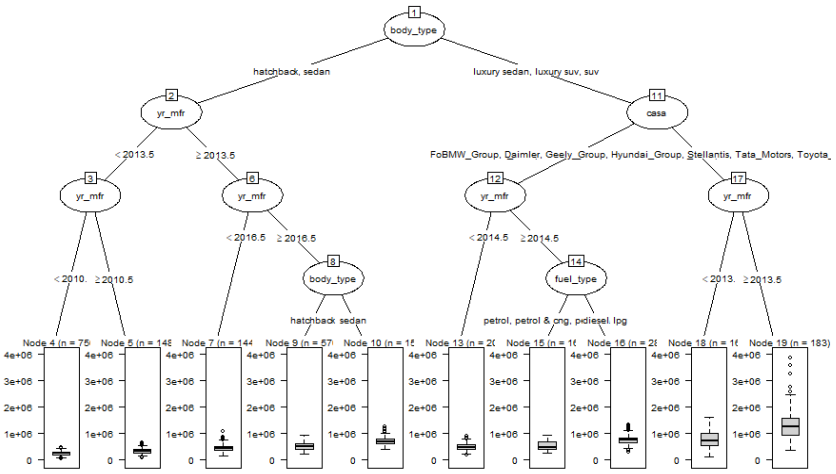
El gráfico muestra en el eje superior el tamaño del árbol al que corresponden los puntos. Los valores de cp mostrados en el gráfico son intermedios de los de la tabla anterior. Se puede observar en general un sobreajuste bastante moderado, ya que la línea roja no se encuentra muy alejada de la negra.

El tamaño óptimo del árbol podría ser de 10 puesto que parece ser el primer valor situado por debajo de la línea horizontal.

Modelo 2 Podado



Como resultado, el modelo podado tiene 10 hojas, con una profundidad de 4. El nodo 8 es el que tiene la media más baja y corresponde a los coches con carrocería hatchback o sedán cuyo año de fabricación es inferior a 2011. Y el nodo con media más alta es el 15, dibujado en azul oscuro, que representa un 3.4% de los datos y representa a los coche con una carrocería diferente a la ya mencionada, una casa diferente a Ford, Honda, Nissan, Otros y Renault y un año de fabricación inferior a 2014.



En este gráfico se nos muestra una representación más exhaustiva del árbol ya que nos añade información a través de los boxplots sobre cómo se distribuyen las observaciones de cada hoja. Con esto, sabremos que nuestra predicción es fiable cuanta menor variabilidad/dispersión tengamos, es decir, cuanto más estrecha sea la caja, más cortos los bigotes, menos outliers (o que estén muy cercanos a los extremos de los bigotes), etc.

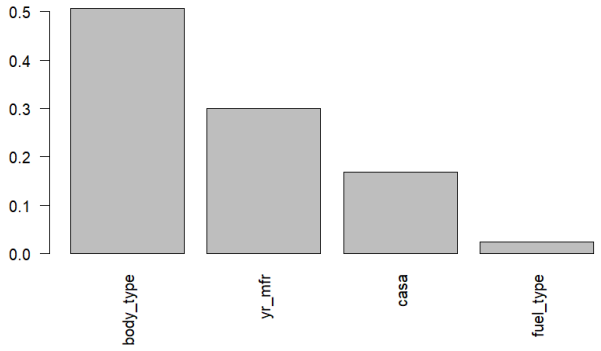
Con esta información podemos ver que las hojas con mejores predicciones es en la que estimamos el menor número de precios de coche, el nodo 4 y el 5, porque como explicamos anteriormente, tienes las cajas más estrechas de todas y además sus outliers están muy agrupados cerca de los bigotes.

Sin embargo, la hoja con peores predicciones es en la que predecían el mayor número de visitantes, que se encuentra más a la derecha, puesto que en este caso ocurre todo lo contrario, su caja es muy amplia y tienen un mayor rango de datos lo que da lugar a mayor posibilidad de fallo a la hora de predecir.

Evaluación del modelo final

El modelo resultante ha obtenido un R2 en train y test de 0.6571219 y 0.6282152, respectivamente. Encontramos poco sobreajuste, ya que hay una diferencia de 2,9 puntos porcentuales entre los valores ya mencionados, con ello podemos intuir que el modelo es bastante estable. El R2 es medianamente alto, por lo que la calidad de sus predicciones son decentes. En comparación con el modelo sin podar, han bajado levemente ambas métricas.

Importancia de las variables



Por último, vamos a ver qué papel tienen las variables dentro del modelo final.

La variable que ha mostrado ser más importante para el modelo ha sido el tipo de carrocería del coche a la hora de predecir los precios, siendo el 50% de información del modelo, el siguiente más importante es el año de fabricación del coche con una diferencia de 20 puntos porcentuales.

2.3 RANDOM FOREST

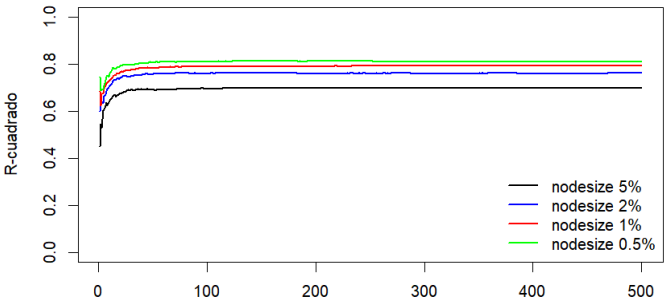
En esta parte del trabajo construiremos un modelo Random Forest con el objetivo de predecir la variable sale_price de forma precisa y estable. Este método se basa en combinar múltiples árboles de decisión entrenados sobre distintas muestras y subconjuntos de variables, lo que permite reducir el sobreajuste y mejorar el rendimiento frente a un único árbol.

El análisis que realizaremos no se limita a ajustar un modelo, sino que busca determinar la configuración óptima del Random Forest. Para ello evaluaremos distintos tamaños de hoja, número de árboles y valores del parámetro **mtry**, comparando su impacto en la capacidad predictiva del modelo mediante métricas.

En definitiva, el objetivo es identificar el Random Forest que mejor se adapta a nuestros datos, justificando todas las decisiones tomadas y analizando también la importancia de las variables dentro del modelo.

Los distintos modelos de Random Forest muestran una mejora progresiva a medida que se reduce el tamaño mínimo de hoja. Con un *nodesize* del 5%, el modelo explica aproximadamente el 70% de la varianza, mientras que al disminuirlo al 0.5% la capacidad explicativa aumenta hasta más del 81%. Esta tendencia refleja que árboles más profundos capturan mejor la estructura del mercado inmobiliario presente en los datos, reduciendo el error y aumentando el porcentaje de varianza explicada.

nodesize <chr>	R2_train <dbl>	R2_test <dbl>
5%	0.7517925	0.7341711
2%	0.8215611	0.7877590
1%	0.8647127	0.8168098
0.5%	0.8987878	0.8358511

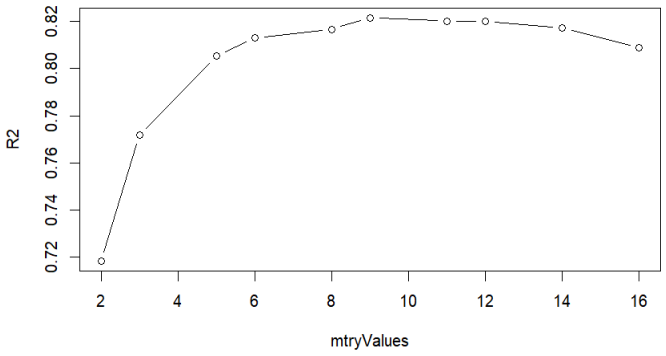


Los resultados muestran cómo el rendimiento del Random Forest mejora progresivamente a medida que se reduce el tamaño mínimo de hoja. Con un *nodesize* del 5%, el modelo presenta un R^2 cercano a 0.75 tanto en entrenamiento como en test, reflejando árboles relativamente simples y una capacidad predictiva moderada. Al disminuir el tamaño de hoja al 2%, el R^2 aumenta hasta valores en torno a 0.82 en entrenamiento y 0.80 en test, lo que indica que árboles algo más profundos permiten capturar mejor la estructura de los datos sin deteriorar la generalización.

Esta tendencia continúa con un *nodesize* del 1%, donde el R^2 alcanza aproximadamente 0.87 en entrenamiento y 0.83 en test, evidenciando que una mayor profundidad incrementa la flexibilidad del modelo y mejora su capacidad explicativa. Finalmente, el mejor rendimiento se obtiene con un *nodesize* del 0.5%, alcanzando un R^2 cercano a 0.90 en entrenamiento y 0.85 en test. Estos resultados sugieren que el bosque es capaz de aprovechar árboles más complejos sin incurrir en un sobreajuste significativo, ya que la diferencia entre entrenamiento y test se mantiene controlada.

Además, el gráfico muestra que el R^2 se incrementa rápidamente en los primeros árboles y se estabiliza a partir de un número relativamente reducido, en torno a 80–100 árboles. A partir de ese

punto, aumentar el número de árboles no produce mejoras apreciables en el rendimiento, sino que únicamente incrementa el coste computacional. En conjunto, los modelos más profundos son los que ofrecen mejor capacidad predictiva, siendo el *nodesize* del 0.5% es el que proporciona el equilibrio más sólido entre ajuste y generalización.



El gráfico de tuning muestra la evolución del R^2 para distintos valores de mtry. El rendimiento aumenta rápidamente en los valores iniciales y alcanza su máximo en torno a mtry = 9. A partir de este punto, la curva se estabiliza y comienza un ligero descenso, lo que indica que valores superiores no aportan mejoras y pueden reducir la aleatoriedad del bosque.

Por tanto, *mtry = 9* se considera el valor óptimo, ya que es el punto con mayor R^2 y a partir del cual el rendimiento comienza a disminuir, cumpliendo el criterio habitual de seleccionar el valor máximo antes de que aparezca la caída de rendimiento.

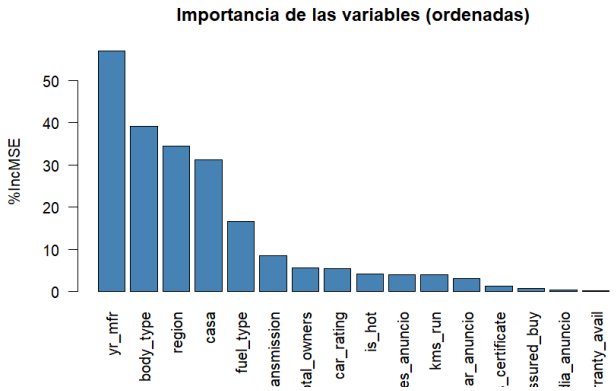
```
Call:
 randomForest(formula = sale_price ~ ., data = datos_train, nodesize = ceiling(0.005 *
nrow(datos_train)), mtry = 9, ntree = 100, importance = TRUE)
               Type of random forest: regression
                     Number of trees: 100
No. of variables tried at each split: 9

          Mean of squared residuals: 12771043356
                    % Var explained: 81.74
```

```
> R2_train_final
[1] 0.9065842
> R2_test_final
[1] 0.8474999
```

El modelo final de Random Forest se ha estimado con 100 árboles, un tamaño mínimo de hoja del 0.5% de las observaciones y un valor de **mtry** igual a 9. Esta configuración permite construir árboles relativamente profundos y aprovechar un conjunto amplio de variables en cada partición, manteniendo la aleatoriedad propia del método. El modelo explica aproximadamente el 81.7% de la variabilidad del precio de venta según la medida interna del bosque, lo que refleja un buen ajuste global.

En cuanto al rendimiento predictivo, el R^2 en el conjunto de entrenamiento es 0.906, mientras que en el conjunto de prueba se sitúa en torno a 0.85. La diferencia entre ambos valores es reducida, lo que indica que el modelo generaliza adecuadamente y no presenta un sobreajuste significativo. En conjunto, el modelo final ofrece un equilibrio sólido entre capacidad de ajuste y rendimiento fuera de muestra, siendo una especificación adecuada para la predicción del precio de venta.



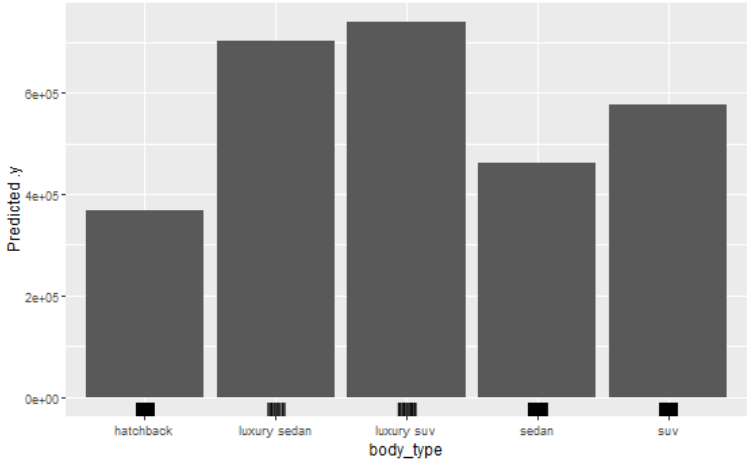
El gráfico muestra claramente que no todas las variables aportan la misma información al modelo. La más influyente es *yr_mfr,* que destaca con mucha diferencia respecto al resto, lo que indica que el año de fabricación es el factor que más contribuye a reducir el error de predicción del precio. A continuación aparecen *body_type, region y casa*, que también tienen un peso relevante, reflejando que las características estructurales del

vehículo y su localización influyen de forma importante en el valor de venta.

En un segundo nivel se sitúan variables como *fuel_type y transmission*, cuyo impacto es moderado pero consistente, aportando información adicional sobre el tipo y uso del vehículo.

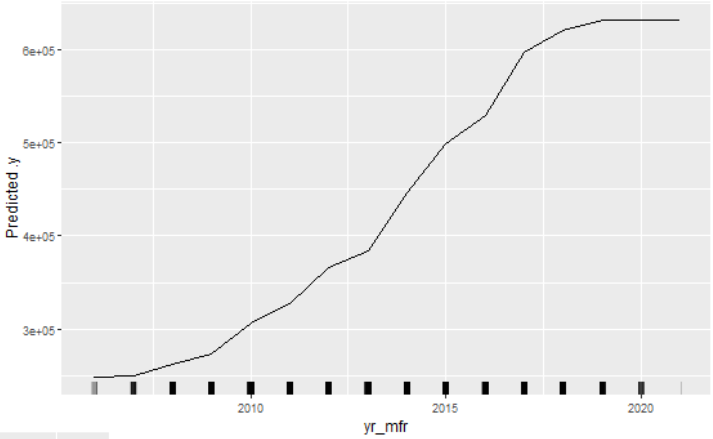
En conjunto, la interpretación es que el precio de venta está dominado por unas pocas variables clave, principalmente relacionadas con la antigüedad y el tipo de vehículo, mientras que otras características tienen un efecto marginal. Esto confirma que el Random Forest ha sabido identificar y priorizar correctamente los factores más determinantes para explicar el precio.que su capacidad explicativa es limitada dentro del modelo.

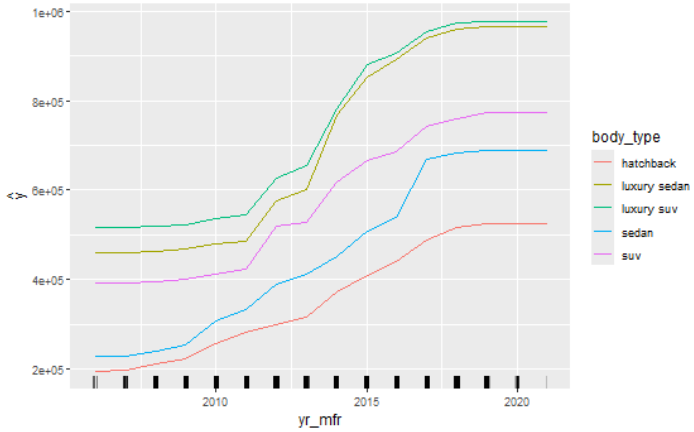
Para finalizar, representaremos las 2 variables más importantes, tanto separadas como juntas, y la variable menos importante mediante los gráficos PDP. Estos gráficos nos permiten entender el efecto promedio de las variables input en la objetivo.



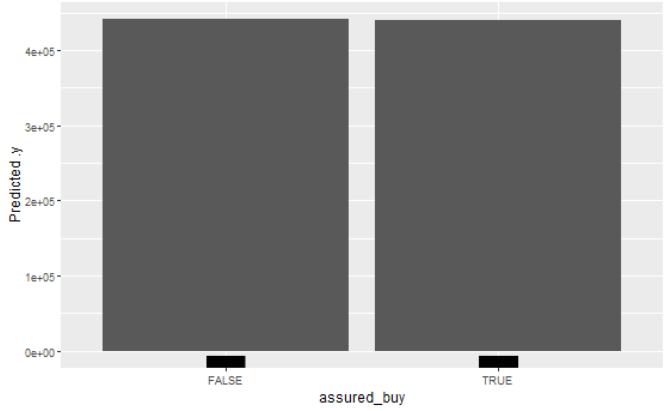
Observamos que existe una jerarquía clara de precios por tipo de carrocería. La mayor diferencia la encontramos entre la carrocería hatchback y la carrocería luxury suv. Esta diferencia asciende a unas 380000 unidades monetarias.

Representamos la evolución del precio de venta en función de los años de fabricación. A medida que el año de fabricación aumenta, la curva muestra una tendencia creciente. Esto nos indica que los coches que se han fabricado en años más recientes tienen un precio de venta mayor.





En todos los tipos de carrocería se observa un crecimiento del precio de venta a medida que aumenta el año de fabricación. En líneas generales, no se aprecian diferencias sustanciales en la evolución temporal entre los distintos tipos de carrocería.



Apenas hay diferencia entre si el coche tiene opción de compra garantizada o no. Esto lógico ya que la importancia de assured_buy era ínfima.

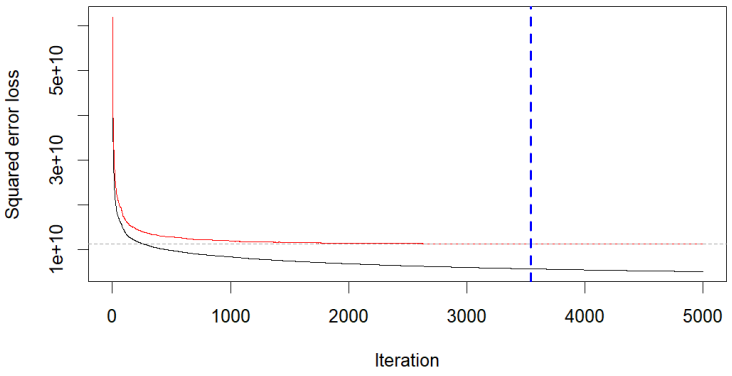
2.4 MODELO GRADIENT BOOSTING

En este apartado vamos a hacer el modelo *Gradient Boosting*, que junto a los *Adaboost* son modelos que construyen árboles de manera secuencial corrigiendo los errores cometidos por los árboles anteriores. De esta manera, se captura nueva información en cada iteración, mientras que en Random Forest se construían de forma independiente.

Como Adaboost es para predecir variables cualitativas, no podremos emplearlo en nuestro trabajo, ya que nuestra variable dependiente (*sale_price*) es cuantitativa continua. Haremos un modelo *Gradient Boosting (GBM)* para minimizar la función *SSE* (ya que es de regresión, sino sería el logaritmo de la función de verosimilitud).
Como no tenemos los parámetros β, para aproximarnos al óptimo haremos un primer árbol y a partir de este, se generarán árboles de forma secuencial con el objetivo de reducir progresivamente el error del modelo. Cada nuevo árbol se ajusta sobre los errores cometidos por los árboles anteriores, buscando aquellas características que hacen que los árboles anteriores no funcionen bien.

Utilizando la función *gbm()* se ha construido un modelo *GB* para regresión con 5000 árboles. Para ello hemos afinado los siguientes parámetros: Hemos utilizado 5000 árboles pero vamos a buscar un número óptimo menor que implique un equilibrio entre un error mínimo y una gran capacidad predictiva, la validación cruzada ha indicado que serían necesarios 3547 árboles y se han utilizado 14 predictoras. La tasa de aprendizaje α es el tamaño de los escalones con los que nos vamos acercando al valor óptimo, toma valores entre 0.001 y 0.3, nosotros hemos escogido 0.2.
En cuanto a los parámetros asociados al tamaño de los árboles, hemos establecido un tamaño mínimo de hoja del 5% y una profundidad de 3.

```
A gradient boosted model with gaussian loss function.
5000 iterations were performed.
The best test-set iteration was 3547.
There were 16 predictors of which 14 had non-zero influence.
```



Con este gráfico representamos la evolución del *SSE* en entrenamiento (línea negra) y en validación (línea roja), la línea vertical azul nos muestra el valor óptimo donde se minimiza el error. Como hemos comentado previamente, la línea azul se encuentra en 3547 que es un número muy alto.

Observamos que a medida que aumenta el número de árboles, el error disminuye más rápido en entrenamiento que en validación. La línea roja se estabiliza bastante a partir de los 1500 árboles, por tanto, creemos que no empeora la calidad predictiva del modelo a partir de dicho punto (y así, evitamos el sobreajuste que aparece con más iteraciones). Además, cuantos menos árboles utilicemos, el proceso iterativo será más rápido y tendrá menor coste computacional.

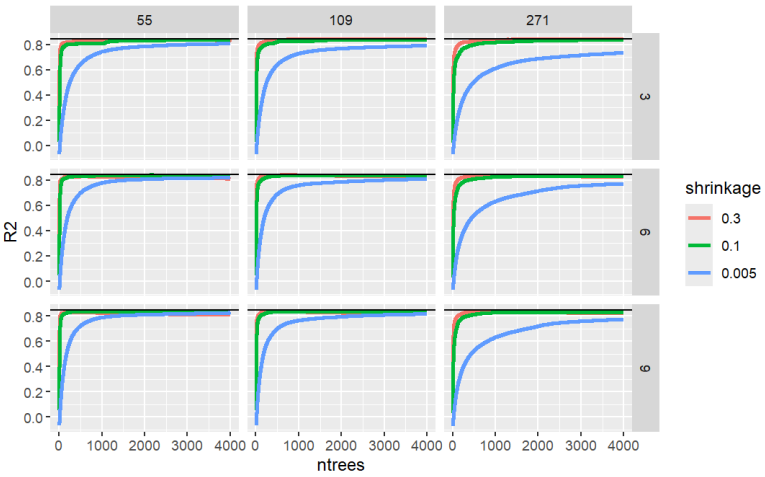
```
# creamos la rejilla
hyper_grid <- expand.grid(
  shrinkage = c(0.3, 0.1, 0.005),
  interaction.depth = c(3, 6, 9),
  n.minobsinnode = c(ceiling(0.01*nrow(datos_train)), ceiling(0.02*nrow(datos_train
)), ceiling(0.05*nrow(datos_train))))
```

Con este código creamos una rejilla para tener 27 combinaciones diferentes en función de la tasa de aprendizaje (*shrinkage*), la profundidad de los árboles (*interaction.depth*) y los distintos tamaños mínimos de la hoja (*n.minobsinnode*).

Así, obtenemos los gráficos y la tabla correspondiente:

shrinkage <fctr>	interaction.dep... <fctr>	n.minobsinnode <fctr>	ntrees <int>	R2 <dbl>
0.1	3	55	3122	0.8376849
0.3	6	271	1014	0.8365547
0.1	9	55	1668	0.8359306
0.1	6	271	3349	0.8352036
0.3	9	55	167	0.8343287
0.3	9	271	810	0.8334448

Si observamos el gráfico, vemos que a mayor tamaño mínimo de hoja, obtenemos peores resultados. La línea azul corresponde a los árboles con una menor tasa de aprendizaje (0.005) y es la que más se aleja del R^2 máximo en todos los casos. En general, los resultados son bastante buenos ya que muchas de las líneas alcanzan la horizontal negra que indica el máximo R^2 (que es 0.8449)




```
optComb<-which((hyper_grid$shrinkage==0.3)&(hyper_grid$interaction.depth==9)&
(hyper_grid$n.minobsinnode==55))
print(tunningGB[[optComb]])
```

Para tomar una decisión, miramos la tabla y buscamos aquel modelo con mayor calidad predictiva y menor número de árboles posible. En nuestro caso, optamos por la combinación de 167 árboles, una tasa de aprendizaje de 0.3, profundidad de 9 y un tamaño mínimo de hoja del 1%. Este modelo alcanza un R^2 de 0.8343, muy cercano al máximo, y creamos ese modelo con los parámetros óptimos:

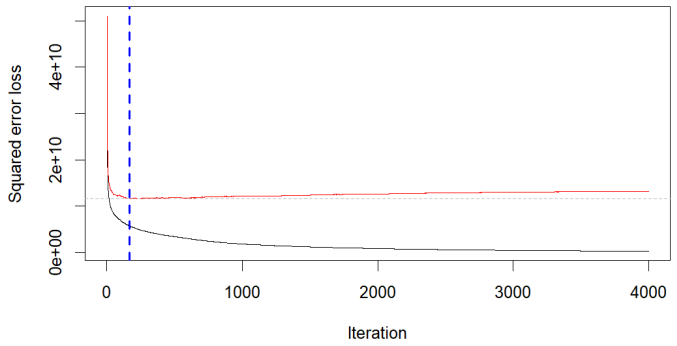
Y obtenemos los siguientes resultados:

```
A gradient boosted model with gaussian loss function.
```

```
4000 iterations were performed.

The best test-set iteration was 167.

There were 16 predictors of which 15 had non-zero influence.
```

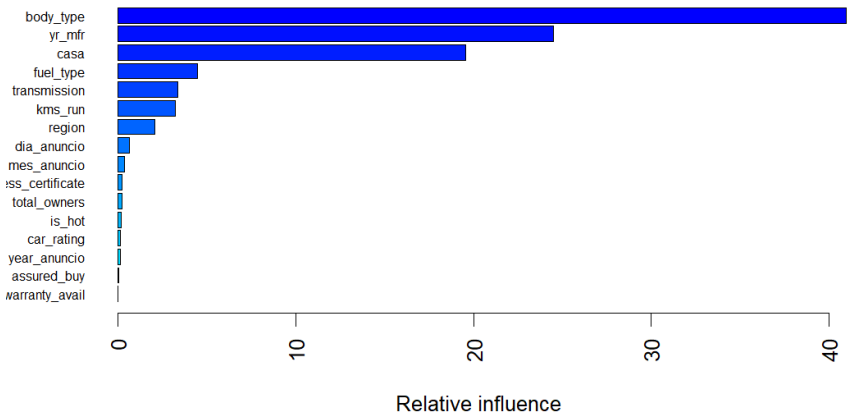


De nuevo, representamos gráficamente la evolución del error y vemos que con 167 árboles alcanzamos el punto óptimo y obtenemos una buena calidad predictiva. Además es una buena opción para reducir el sobreajuste.

Para evaluar el modelo, calcularemos el R^2 en entrenamiento y prueba y obtenemos los siguientes resultados:

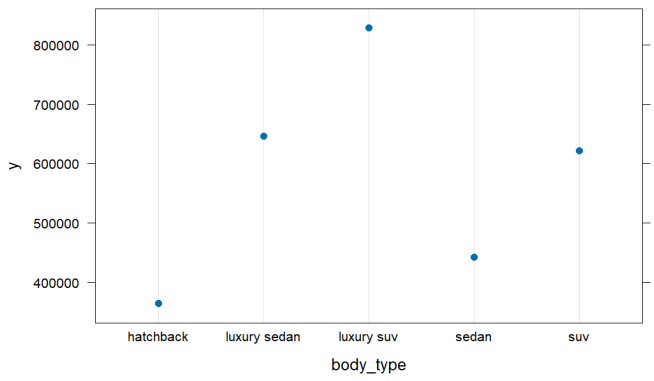
- Entrenamiento: `0.9150686`
- Prueba: `0.8621249`

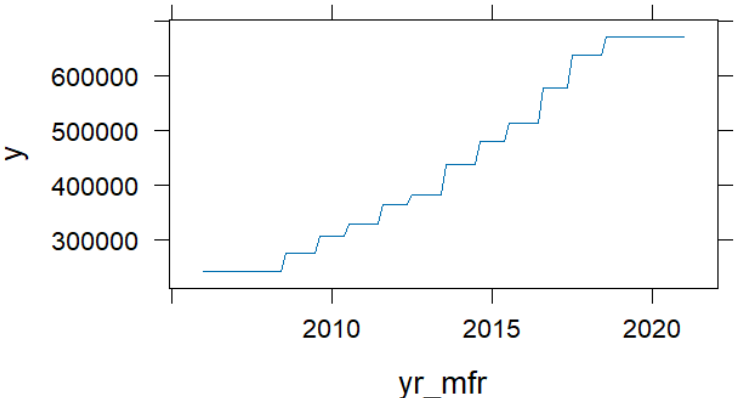
A diferencia de otros modelos, hemos obtenido una calidad predictiva bastante buena, con un R^2 del 0.86%. Observamos que hay un ligero sobreajuste, pero no indica una gran falta de estabilidad en nuestro último modelo.
Por último, haremos un análisis de las variables, empezando por un ranking de variables:



La variable más importante es el tipo de carrocería del coche con un 40% de información, seguida del año de fabricación del coche, la marca del fabricante del motor y el tipo del combustible.

Por otro lado, para obtener más información sobre la influencia de estas variables sobre la objetivo, recurriremos a los gráficos de dependencia parcial (PDP). Empezamos haciendo uno para una predictora categórica y otro para una continua.

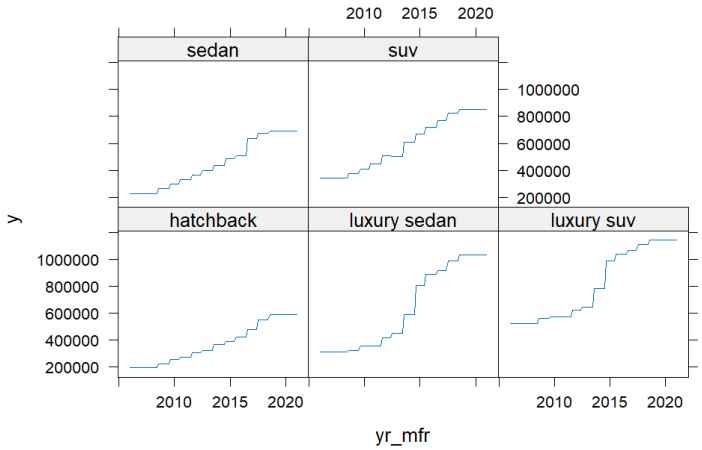




En el primer gráfico, observamos el efecto marginal de la variable explicativa más importante, que nos muestra el valor medio predicción según el tipo de carrocería del coche. Observamos que los *luxury SUV* presentan el valor predicho más alto, mientras que *hatchback* muestra el valor medio más bajo. Además, observamos que el eje Y tiene un rango de valores amplio, por lo que *body_type* implica variaciones en el precio ya que la diferencia entre los niveles más extremos es de 400.000.

Por otro lado, observamos que el efecto del año de fabricación del coche tiene una tendencia creciente. A medida que aumenta el año de fabricación, el valor medio predicho por el modelo también aumenta. Por tanto, los vehículos más nuevos tienden a asociarse con valores más altos de la variable objetivo.

Por último, haremos el gráfico con las dos variables juntas (*yr_mfr* y *body_type*):



En todos los tipos de carrocería se observa una tendencia creciente entre el año de fabricación y la variable objetivo. Sin embargo, el rango de los valores de las predicciones varían según el tipo de vehículo.

Los vehículos de alta gama (*luxury sedan* y *luxury suv*) presentan valores medios más elevados y un rango más amplio de valores durante el periodo.

En cambio, los coches con carrocería *hatchback* y *sedan* tienen un crecimiento más moderado y unos niveles predichos más bajos.

FASE 3: CONCLUSIÓN

El objetivo de nuestro trabajo es realizar un análisis riguroso que pueda ser utilizado como herramienta de apoyo a la toma de decisiones de una empresa como puede ser un concesionario.

Vamos a comparar las métricas de los diferentes modelos para obtener una respuesta adecuada y decidir qué es lo mejor que podemos ofrecer al concesionario.

Regresión modelo StepBic: Obtuvimos un R^2 en train de 0.703 y en test de 0.704, sin sobreajuste.

Árbol podado: En el árbol podado resultante, el R^2 en train es de 0.657 y el de test 0.628. Con sobreajuste muy reducido.

Random Forest: Para este modelo, alcanzamos un R^2 en train de 0.906 y en test de 0.85. Sobreajuste de 0.05 puntos pero un R^2 bastante mejor que en los casos anteriores tanto en train como en test.

Gradient Boosting: En el modelo secuencial, logramos un R^2 en train de 0.915 y en test de 0.862. Este tipo de modelos tienen tendencia al sobreajuste, por lo que la diferencia puede ser mayor de lo normal y no deberíamos confiar en el valor de entrenamiento. De todas maneras, la capacidad predictiva es bastante buena.

Haciendo uso de los resultados obtenidos y teniendo en cuenta el objetivo del estudio, concluimos que el modelo Gradient Boosting es el modelo que seleccionaremos como la herramienta predictiva que genera resultados más fiables y robustos de cara a un concesionario, puesto que estos modelos son mucho más sofisticados de cara a las predicciones.

Esto se fundamenta en el propio algoritmo del modelo ya que como sabemos, estos modelos boosting se centran en corregir de manera secuencial los residuos producidos por los árboles anteriores, lo que da lugar a predicciones más exactas. Esto es un aspecto crucial de cara a la toma de decisiones como la optimización del inventario o el margen de beneficio, que se llevan a cabo en un concesionario y que se basarán en nuestros resultados por lo que se buscará priorizar la capacidad predictiva frente a otras opciones como la complejidad del modelo.